

You can build a terminal-level tool to recursively search for a string in a project folder using **awk**,

■ 1. Easiest: Use Unix tools (grep + awk)

If your goal is just **recursive string search**, you don't even need Perl or Python.

```
```bash
grep -rn "your_string" /path/to/project

* -r` → recursive
* -n` → show line numbers

Add awk for filtering/formatting:

```bash
grep -rn "your_string" . | awk -F: '{print "File: \"$1\", Line: \"$2\", Match: \"$3\"}'
```

- Best for: quick terminal use
- Limited customization

■ 2. Using awk alone (not ideal for recursion)

``awk`` itself **does not handle recursion well** – you'd need ``find``:

```
```bash
find . -type f -exec awk '/your_string/ {print FILENAME ":" NR ":" $0}' {} +
```

- Works, but messy
- ➔ ■ awk is better for processing, not directory traversal.

\*\*\*

# ■ 3. Perl (very powerful for this)

Perl is excellent for file scanning and recursion.

### Example:

```
```perl
use File::Find;

my $search = "your_string";

find(sub {
    return unless -f;
    open my $fh, '<', $_ or return;

    while (<$fh>) {
        if (/ $search/) {
            print "$File::Find::name:$.: $_";
        }
    }

    close $fh;
}, '.');
```

Run:

```
```bash
perl search.pl
```

- Pros:
- \* Fast

```
* Regex powerhouse
* Built-in recursion (`File::Find`)
```

```

```

#### # ■ 4. Python (best for building a full app)

If you want a real terminal tool, Python is the best choice.

### Basic recursive search:

```
```python
import os

search = "your_string"

for root, dirs, files in os.walk("."):
    for file in files:
        filepath = os.path.join(root, file)
        try:
            with open(filepath, "r", errors="ignore") as f:
                for i, line in enumerate(f, 1):
                    if search in line:
                        print(f"{filepath}:{i}: {line.strip()}")
        except:
            pass
```
```

Run:

```
```bash
python search.py
```
```

```

```

#### # ■ 5. Make it a real terminal app (Python CLI)

You can enhance it with arguments:

```
```python
import os
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("term")
parser.add_argument("--path", default=".")
args = parser.parse_args()

for root, dirs, files in os.walk(args.path):
    for file in files:
        path = os.path.join(root, file)
        try:
            with open(path, "r", errors="ignore") as f:
                for i, line in enumerate(f, 1):
                    if args.term in line:
                        print(f"{path}:{i}: {line.strip()}")
        except:
            continue
```
```

Usage:

```
```bash
python search.py "hello" --path ./project
```
```

```

```

#### # ■ 6. Combining tools (Power combo)

You can combine them for efficiency:

### Example:

```
```bash
find . -type f | xargs perl -ne 'print "$ARGV$.: $_" if /your_string/'
```
```

\*\*\*

# ■ Recommendation (What you should use)

| Goal                | Best Tool           |
|---------------------|---------------------|
| Quick search        | <code>`grep`</code> |
| Text processing     | <code>`awk`</code>  |
| Regex-heavy scripts | Perl                |
| Build a CLI app     | Python ■            |

\*\*\*

# ■ Pro Tip

If your goal is real productivity, use:

```
```bash
ripgrep (rg)
```
```

```
```bash
rg "your_string"
```
```

It's:

- \* Much faster than grep
- \* Ignores ``.gitignore``
- \* Built for dev workflows

\*\*\*

# ■ Final Answer

■ Yes, you can use awk, Perl, or Python – but:

- \* `**awk alone is not ideal for recursion**`
- \* `**Perl is great for scripting**`
- \* `**Python is best for building a full terminal application**`

\*\*\*

We can build a `**small practical “agent-style” Python CLI app**` that:

- \* Recursively scans a project (like ``grep``)
- \* Uses `**awk-like filtering**`
- \* Uses `**Perl-style regex**`
- \* Extracts suspicious code (e.g., errors, TODOs, logs)
- \* Prepares output you can feed into an `**LLM for debugging**`

\*\*\*

# ■ ■ “CodeBugAgent” – Minimal Terminal Debug Assistant

This is a single Python script you can run from terminal.

\*\*\*

## ■ Features

- \* Recursive file search
- \* Regex (Perl-like patterns)
- \* awk-style filtering of results
- \* Detect common bug indicators
- \* Export results for LLM input

```

■ Python App

```python
#!/usr/bin/env python3

import os
import re
import argparse

# --- Patterns (Perl-style regex)
BUG_PATTERNS = [
    r"error",
    r"exception",
    r"fail(ed)?",
    r"traceback",
    r"undefined",
    r"null",
    r"TODO",
    r"FIXME"
]

def scan_file(filepath, pattern, ignore_case=True):
    results = []
    flags = re.IGNORECASE if ignore_case else 0

    try:
        with open(filepath, "r", errors="ignore") as f:
            for lineno, line in enumerate(f, 1):
                if re.search(pattern, line, flags):
                    results.append((filepath, lineno, line.strip()))
    except Exception:
        pass

    return results

def recursive_scan(root, pattern):
    all_results = []

    for dirpath, _, filenames in os.walk(root):
        for filename in filenames:
            filepath = os.path.join(dirpath, filename)
            matches = scan_file(filepath, pattern)
            all_results.extend(matches)

    return all_results

# --- awk-like filter
def awk_filter(results, keyword=None):
    if not keyword:
        return results

    return [r for r in results if keyword.lower() in r[2].lower()]

# --- Detect bug hints
def detect_bugs(root):
    findings = []
    combined_pattern = "|".join(BUG_PATTERNS)
    return recursive_scan(root, combined_pattern)

# --- Format output for LLM
def format_for_llm(results, limit=20):
    formatted = []
    for file, line, text in results[:limit]:
        formatted.append(f"{file}:{line} -> {text}")

    return "\n".join(formatted)

```

```

def main():
    parser = argparse.ArgumentParser(description="CodeBugAgent CLI")
    parser.add_argument("path", help="Path to project")
    parser.add_argument("--search", help="Search string (regex supported)")
    parser.add_argument("--filter", help="awk-style filter keyword")
    parser.add_argument("--bugs", action="store_true", help="Detect bug patterns")
    parser.add_argument("--llm", action="store_true", help="Format for LLM")

    args = parser.parse_args()

    results = []

    if args.bugs:
        print("■ Detecting potential bugs...")
        results = detect_bugs(args.path)

    elif args.search:
        print(f"■ Searching for: {args.search}")
        results = recursive_scan(args.path, args.search)

    if args.filter:
        results = awk_filter(results, args.filter)

    if args.llm:
        print("\n■ LLM Input:\n")
        print(format_for_llm(results))
    else:
        for file, line, text in results:
            print(f"{file}:{line}: {text}")

    print(f"\n■ Found {len(results)} matches")

```

```

if __name__ == "__main__":
    main()

```

■ How to Use

1. Save file:

```

```bash
nano codebugagent.py
```

```

2. Run:

```

```bash
python3 codebugagent.py .
```

```

■ Example Commands

■ Search like grep:

```

```bash
python codebugagent.py . --search "database error"
```

```

■ Find bugs automatically:

```

```bash
python codebugagent.py . --bugs
```

```

■ awk-like filtering:

```
```bash
python codebugagent.py . --bugs --filter "null"
```
```

■ Prepare output for LLM:

```
```bash
python codebugagent.py . --bugs --llm
```
```

Example output:

```
src/db/conn.py:45 -> connection error occurred
app/main.py:12 -> TODO fix login issue
```

■ How this integrates with LLM

You can copy this output directly into an LLM like:

These are suspicious log lines from my project:

<PASTE OUTPUT>

Help me debug root cause.

■ Why this works (architecture)

| Component | Role |
|-------------------|---------------------|
| Python | Main agent |
| Regex (Perl-like) | Pattern matching |
| awk-style filter | Stream filtering |
| os.walk | Recursive traversal |
| Formatting layer | LLM integration |

■ Final Takeaway

■ Yes, you *can* build a lightweight debugging agent combining:

- * Python (control + CLI)
- * Perl-style regex (powerful search)
- * awk-like filtering (post-processing)

And this script is a *real starting point* you can use today.

Yes ■ – what you're thinking is *exactly the right architecture* for using LLMs efficiently in real-world

■ *Short answer:*

- > Yes, you should absolutely pre-process large GitLab log files locally (with your Python agent) and send them to the LLM.
- > This is a *best practice* to reduce token usage, cost, and noise.

■ ■ Recommended Architecture (What you're proposing)

```
GitLab Logs (Huge file)
    ↓
Python Agent (your script)
    - parse logs
```

- extract error blocks
- deduplicate
- summarize

↓

Clean small context

↓

LLM (Windsurf / GPT / etc.)

↓

Bug analysis + fix suggestions

■ This is exactly how production AI debugging pipelines work.

■ Upgrade your agent for LOG PROCESSING

Let's enhance your earlier script to:

```
* Handle **large log files efficiently**
* Extract only **error sections**
* Capture **context lines**
* Output **LLM-ready chunks**
```

■ ■ Improved Python Log Analyzer Agent

```
```python
#!/usr/bin/env python3
```

```
import argparse
import re
```

```
ERROR_PATTERNS = [
 r"ERROR",
 r"Exception",
 r"Traceback",
 r"FATAL",
 r"FAIL",
 r"CRITICAL"
]
```

```
def extract_errors(file_path, context=3):
 results = []
 pattern = re.compile("|".join(ERROR_PATTERNS), re.IGNORECASE)

 with open(file_path, "r", errors="ignore") as f:
 lines = f.readlines()

 for i, line in enumerate(lines):
 if pattern.search(line):
 start = max(0, i - context)
 end = min(len(lines), i + context + 1)

 snippet = lines[start:end]
 results.append("".join(snippet))

 return results
```

```
def deduplicate(snippets):
 seen = set()
 unique = []

 for s in snippets:
 key = s.strip()
 if key not in seen:
 seen.add(key)
 unique.append(s)
```

```
 return unique
def limit_output(snippets, max_blocks=10):
```

```

 return snippets[:max_blocks]

def format_for_llm(snippets):
 output = "Analyze the following log errors and suggest root cause:\n\n"

 for i, s in enumerate(snippets, 1):
 output += f"--- Error Block {i} ---\n{s}\n"

 return output

def main():
 parser = argparse.ArgumentParser(description="Log Error Extractor Agent")
 parser.add_argument("file", help="Path to log file")
 parser.add_argument("--context", type=int, default=3)
 parser.add_argument("--limit", type=int, default=5)
 parser.add_argument("--llm", action="store_true")

 args = parser.parse_args()

 errors = extract_errors(args.file, args.context)
 errors = deduplicate(errors)
 errors = limit_output(errors, args.limit)

 if args.llm:
 print(format_for_llm(errors))
 else:
 for e in errors:
 print("-----\n", e)

 print(f"\n■ Extracted {len(errors)} error blocks")

if __name__ == "__main__":
 main()

■■ Usage Examples

■ Extract errors from GitLab log:

```bash
python log_agent.py gitlab.log
```

■ Send only useful data to LLM:

```bash
python log_agent.py gitlab.log --llm
```

Output:

 Analyze the following log errors and suggest root cause:

 --- Error Block 1 ---
 Traceback (most recent call last):
 ...
 Connection refused

 --- Error Block 2 ---
 ERROR: failed to connect DB
 ...

➡ ■ ■ Perfect for Windsurf agent input

```



```

■ ■ Why this is IMPORTANT (LLM optimization)

■ Bad method:

* Send entire log file (100MB)
* Token overflow
* Cost explosion
* LLM gets confused

■ Your method:

* Extract ~1-5% useful lines
* Higher signal, lower noise
* Much better debugging

■ ■ Advanced Improvements (Highly Recommended)

You can extend this agent further:

■ 1. Stack trace grouping

Group multi-line exceptions:

```python
if "Traceback" in line:
    # capture until empty line
```

■ 2. Error clustering

Group similar errors → avoid repetition

■ 3. Time-based filtering

```python
# only last 1 hour logs
```

■ 4. Severity filtering

```bash
--level ERROR
--level CRITICAL
```

■ 5. Direct LLM integration (future)

```python
import openai

response = openai.ChatCompletion.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
)
```

■ ■ Bonus: Combine with your earlier code search agent

```

You can now do:

Step 1 → Extract log errors  
Step 2 → Search codebase for those errors  
Step 3 → Send BOTH to LLM

Example:

Error: "DB connection refused"  
+  
Matching code from repo  
→ Much better debugging context

\*\*\*

# ■ Final Answer

■ Yes – not only *\*can\** you do this, you *\*\*should do this\*\**.

This is:

\* ■ Efficient  
\* ■ Scalable  
\* ■ Production-grade approach

\*\*\*